UNIT 5

FUNCTIONS

INTRODUCTION

In C++, functions are like mini-programs within your main program. Each function tackles a specific job, like calculating an area or printing a message. Think of them as separate tools in your coding toolbox, each designed for a specific task. By using functions, you can break down complex problems into smaller, easier-to-manage steps, making your code cleaner and more organized. Let's explore how functions work in C++ and how they can make your programming journey smoother.

PRE-DEFINED FUNCTIONS IN C++

In C++, you don't always have to write everything from scratch. Instead, you can leverage the power of pre-defined functions that are already built into the language. These functions perform various tasks, saving you time and effort in your coding endeavors.
Think of them as helpful tools in your C++ toolbox, ready to be used whenever you need a specific task completed. Let's explore some common pre-defined functions and how they can be incorporated into your programs!

USER-DEFINED FUNCTIONS

While pre-defined functions offer a variety of functionalities, sometimes you'll need to create your own custom tools. That's where user-defined functions come in. These are functions that you, the programmer, define to perform specific tasks within your program.
Here are the key components of user-defined functions:

I. FUNCTION DECLARATION

This acts like an announcement, telling the compiler about the existence of your function without providing its implementation details. It includes the function name, parameter list (if any), and return type (optional) written in this format:

C++

```
return_type function_name(parameter_list);
```

II. FUNCTION DEFINITION

This is where the magic happens! You define the actual code your function will execute when called. It includes the function declaration again, followed by the function body enclosed in curly braces {}. The body contains the statements responsible for completing the function's task.

III. FUNCTION CALL

This is how you "use" your function to perform its task. You write the function name followed by parentheses containing the actual values (arguments) you want to pass, if the function requires any.

Example in C++:

C++

```
// Function declaration
double calculate_area(double length, double width);
// Function definition
double calculate_area(double length, double width) {
    return length * width;
}
// Function call (assuming variables length and width are defined with
```

values)
double rectangle_area = calculate_area(length, width);

Function Definition vs. Function Call

Feature	Function Definition	Function Call
Purpose	Defines the function's behavior and code	Executes the defined function's code
Location	Inside the program body	Anywhere in the program body (after declaration)
Arguments	Formal parameters (placeholders)	Actual values passed to the function
Return Value	Defined return type (optional)	Receives the return value (if any)

C++

```
#include <iostream>
using namespace std;
int add_numbers(int a, int b) {
  return a + b;
}
int main() {
  int num1, num2, sum;
  cout << "Enter two numbers: ";
  cin >> num1 >> num2;
  sum = add_numbers(num1, num2);
  cout << "The sum is: " << sum << endl;
  return 0;
}
```

Example Program (Pre-defined Function):

C++

```
#include <iostream>
using namespace std;
int main() {
  int number;
  cout << "Enter a number: ";
  cin >> number;
  cout << "The square of " << number << " is: " << number * number << endl;
  return 0;
}
```

User-defined Functions vs. Pre-defined Functions

Feature	User-defined Functions	Pre-defined Functions
Origin	Created by the programmer	Already defined in the C++ library
Declaration	Required	Not required
Definition	Required	Not required, already defined
Flexibility	Can be customized	Limited customization options
Examples	calculate_area(), add_numbers()	cout(), cin(), sqrt()

LOCAL AND GLOBAL VARIABLES

Variables are fundamental to storing and manipulating data in your programs. expand_more However, their scope, or the extent of their visibility, can be crucial.

Local Variables:
These variables are declared within a function's body. They are only accessible within that specific function and are destroyed when the function finishes execution.

Global Variables:
These variables are declared outside of any function, typically at the beginning of your program. They are accessible from anywhere within the program, but their use should be cautious due to potential naming conflicts and unintended side effects.